RBF Finite-Difference Discretization of Anisotropic Diffusion: Theory and Implementation

Dennis Corraliza

6/21/2026

Abstract

This note derives the Radial Basis Function Finite Difference (RBF-FD) method for the anisotropic diffusion equation $-\nabla \cdot (A\nabla u) = f$ on a bounded domain $\Omega \subset \mathbb{R}^2$, with mixed Dirichlet/Neumann boundary conditions, and shows precisely how each step of the derivation corresponds to a function in the accompanying Python implementation (modules `assembly.py`, `domain.py`, `rbf.py`, `boundary.py`, `stencils.py`, and `geometry.py`).

Contents

1 Problem statement

We seek $u : \Omega \to \mathbb{R}$ satisfying the anisotropic diffusion equation

$$-\nabla\cdot\big(A\nabla u\big) = f \quad \text{in } \Omega, \tag{1}$$

where $A \in \mathbb{R}^{2\times 2}$ is a constant, symmetric positive-definite diffusion tensor and f is a prescribed source term. The boundary $\partial\Omega$ is partitioned into Dirichlet and Neumann pieces:

$$u = g_D \quad \text{on } \Gamma_D \subset \partial\Omega, \tag{2}$$

$$\frac{\partial u}{\partial n} \equiv n \cdot \nabla u = g_N \quad \text{on } \Gamma_N \subset \partial\Omega, \tag{3}$$

where n is the outward unit normal. When $A = I$, (1) reduces to the standard Poisson equation $-\Delta u = f$.

In the code, the geometric domain is either a square $[0, L]^2$ or a disk of radius R (`geometry.py`), the tensor A is supplied directly by the user, and the boundary type on each side/arc is encoded by the list `btype` (e.g. `['dirichlet', 'neumann', 'dirichlet', 'neumann']` for a square, ordered counterclockwise starting at $y = 0$).

2 Radial basis function interpolation

2.1 Local RBF interpolant

Let $\{\mathbf{x}_j\}_{j=1}^N$ be the full node set discretizing Ω. For a fixed node $\mathbf{x}_i$, let $S_i = \{j_1, \dots, j_n\} \subset \{1, \dots, N\}$ index its *stencil*: the n nearest neighbors of $\mathbf{x}_i$ (including $\mathbf{x}_i$ itself), determined by a k-nearest-neighbor search. We build a local interpolant of a function u over the stencil using a radial kernel φ:

$$u(\mathbf{x}) \;\approx\; \sum_{j\in S_i} \lambda_j\, \varphi(\|\mathbf{x} - \mathbf{x}_j\|) \;+\; \sum_{\ell=1}^{m} \mu_\ell\, p_\ell(\mathbf{x}), \tag{4}$$

where $\{p_\ell\}_{\ell=1}^m$ is an (optional) polynomial basis used to *augment* the kernel interpolant, with coefficients constrained by

$$\sum_{j\in S_i} \lambda_j\, p_\ell(\mathbf{x}_j) = 0, \qquad \ell = 1, \dots, m. \tag{5}$$

Two kernels are used in the implementation (`rbf.py`):

$$\varphi_{\text{cubic}}(\mathbf{p}) = \|\mathbf{p}\|^3, \qquad\qquad \varphi_{\text{gauss}}(\mathbf{p}) = \exp\big(-(\varepsilon\|\mathbf{p}\|)^2\big), \tag{6}$$

where $\mathbf{p} = \mathbf{x} - \mathbf{x}_j$ is a displacement vector and ε is a user-chosen shape parameter for the Gaussian. The polynomial basis used for augmentation is linear,

$$\{p_1, p_2, p_3\}(x, y) = \{1,\ x,\ y\}, \tag{7}$$

so that the interpolant exactly reproduces constants and linear functions whenever augmentation is enabled.

Code correspondence.

- `rbf.phi_cubic(p)`, `rbf.phi_gauss(p, eps)` $\leftrightarrow$ (6).
- `rbf.poly_basis(p)` $\leftrightarrow$ (7).
- `stencils.knn_list(P, num_stencil_nodes)` builds S_i for every node.
- `domain.PDEDomainContext.augmentation` is the boolean flag toggling the second sum in (4).

3 RBF-FD differentiation weights

3.1 The linear functional viewpoint

RBF-FD does not differentiate the interpolant (4) symbolically; instead, it seeks weights w_j such that a linear operator $\mathcal{L}$ applied at $\mathbf{x}_i$ is approximated directly as a weighted sum of *nodal values*:

$$(\mathcal{L}u)(\mathbf{x}_i) \;\approx\; \sum_{j \in S_i} w_j \, u(\mathbf{x}_j). \tag{8}$$

The weights are chosen so that (8) is *exact* whenever u is any kernel function centered at a stencil node, and (if augmentation is used) exact for every polynomial in the augmentation basis. This yields the linear system

$$\underbrace{\begin{bmatrix} M & P \\ P^\top & 0 \end{bmatrix}}_{\text{generalized Gram matrix}} \begin{bmatrix} \mathbf{w} \\ \boldsymbol{\mu} \end{bmatrix} = \begin{bmatrix} \mathbf{b} \\ \mathbf{0} \end{bmatrix}, \tag{9}$$

where

$$M_{jk} = \varphi(\mathbf{x}_j - \mathbf{x}_k), \qquad j,k \in S_i, \tag{10}$$

$$P_{j\ell} = p_\ell(\mathbf{x}_j), \qquad j \in S_i,\ \ell = 1,\dots,m, \tag{11}$$

$$b_j = (\mathcal{L}\varphi)(\mathbf{x}_i - \mathbf{x}_j), \qquad j \in S_i. \tag{12}$$

The vector $\mathbf{b}$ holds the operator $\mathcal{L}$ applied *exactly* (in closed form) to the kernel, evaluated at the displacement from the stencil center to each stencil node, rather than any discrete approximation. Solving (9) for $\mathbf{w}$ gives the desired differentiation weights at node $\mathbf{x}_i$.

Code correspondence. This is exactly what `assembly.local_weights_solve(context,i)` and `assembly.local_grad_solve(context,i)` implement, for $\mathcal{L} = \nabla\cdot(A\nabla\,\cdot)$ and $\mathcal{L} = \nabla$ respectively:

- The matrix `M` in the code is exactly M in (12), built from `context.phi`.
- The vector `b` (or matrix `b_grad`, with one column per spatial dimension) is built from `context.laplacian_phi` (resp. `context.grad_phi`), which are the closed-form expressions $\nabla\cdot(A\nabla\varphi)$ and $\nabla\varphi$ derived in Section 4 below.
- The block matrix `Pmat` and the block-augmented system built via `np.block` are exactly the augmented system in (9) when `context.augmentation` is `True`.
- `np.linalg.solve(M, b)` solves (9) and the function returns only the first `num_nodes` entries of the solution, i.e. $\mathbf{w}$ (discarding the polynomial Lagrange multipliers $\boldsymbol{\mu}$).

3.2 Least-squares variant

An alternative formulation avoids solving the square system (9) directly and instead seeks weights that best reproduce $\mathcal{L}$ in a least-squares sense over a richer set of *auxiliary evaluation centers* $\{\mathbf{c}_k\}_{k=1}^{K}$ scattered around $\mathbf{x}_i$ (typically more centers than stencil nodes, $K > n$):

$$\min_{\mathbf{w}} \left\| \underbrace{\big[\varphi(\mathbf{c}_k - \mathbf{x}_j)\big]_{k,j}}_{=:M\in\mathbb{R}^{K\times n}} \mathbf{w} \;+\; \underbrace{\big[p_\ell(\mathbf{x}_j)\big]_{l,j}}_{=:P^T\in\mathbb{R}^{m\times n}} \mathbf{v} \;-\; \underbrace{\big[(\mathcal{L}\varphi)(\mathbf{x}_i - \mathbf{c}_k)\big]_{k}}_{=:\mathbf{b}\in\mathbb{R}^{K}} \right\|_2^2, \tag{13}$$

subject to the same polynomial reproduction constraints (5) penalized in the least squares objective. The motivation for this variant is robustness: anchoring the fit to a denser, more uniformly distributed cloud of auxiliary points reduces sensitivity to clustering or anisotropy in the stencil node positions themselves.

The auxiliary centers $\{\mathbf{c}_k\}$ are generated as a near-uniform polar grid of *num_rings* concentric rings (spacing $h = r/num\_rings$, with r the stencil radius) plus a center point, with the number of angular samples per ring chosen so that the arc-length spacing on each ring is approximately h as well:

$$\mathbf{c}_k = \big(\rho_i \cos\theta_{ij},\ \rho_i \sin\theta_{ij}\big), \qquad \rho_i = ih,\ \ \theta_{ij} = \frac{2\pi j}{n_i},\quad n_i = \max\Big(1,\ \mathrm{round}\big(\tfrac{2\pi\rho_i}{h}\big)\Big). \tag{14}$$

Code correspondence.

- `rbf.generate_grid_2d(r, k)` implements (14), using `rbf.rad_to_euc` to convert each (ρ_i, θ_{ij}) to Cartesian coordinates.
- `assembly.local_weights_ls(context,i)` and `assembly.local_grad_ls(context,i)` build M and $\mathbf{b}$ exactly as in (13), append the polynomial constraint rows, and call `np.linalg.lstsq` to solve.
- Dispatch between the direct system (9) and the least-squares system (13) is controlled by `context.center_rings`: `None` selects the direct solve, any integer value selects the least-squares variant with that many rings *num_rings*.

4 The anisotropic diffusion operator applied to each kernel

The entries b_j in (12) require a closed-form expression for $\nabla\cdot(A\nabla\varphi)$, evaluated at the displacement $\mathbf{p} = \mathbf{x}_i - \mathbf{x}_j$. Writing $A = \left(\begin{smallmatrix} A_{11} & A_{12} \\ A_{12} & A_{22} \end{smallmatrix}\right)$ and $r = \|\mathbf{p}\|$, $\mathbf{p} = (x, y)$:

Cubic kernel. For $\varphi(\mathbf{p}) = r^3$,

$$\nabla\cdot\big(A\nabla\varphi\big)(\mathbf{p}) = 3r\left(\mathrm{tr}(A) + \frac{\mathbf{p}^\top A\mathbf{p}}{r^2}\right). \tag{15}$$

This expression is regular as $r \to 0$ in the sense that the true value of the operator at $\mathbf{p} = 0$ is 0; the implementation returns 0 directly whenever r falls below a tolerance, avoiding the removable $1/r^2$ singularity in (15) without altering the limiting value.

Gaussian kernel. For $\varphi(\mathbf{p}) = \exp(-(\varepsilon r)^2)$,

$$\nabla\cdot\big(A\nabla\varphi\big)(\mathbf{p}) = 2\varepsilon^2\,\varphi(\mathbf{p})\Big(2\varepsilon^2\,\mathbf{p}^\top A\mathbf{p} - \operatorname{tr}(A)\Big). \tag{16}$$

No singularity occurs here since φ and its derivatives are smooth everywhere.

Polynomial augmentation basis. Since $\nabla\cdot(A\nabla p_\ell) \equiv 0$ for $p_\ell \in \{1, x, y\}$ (all second derivatives vanish), the corresponding entries in the augmented right-hand side of (9) are identically zero, independent of A.

Gradients. The two kernel gradients used to build Neumann boundary rows (Section 5.3) are

$$\nabla\varphi_{\text{cubic}}(\mathbf{p}) = 3r\,\mathbf{p}, \qquad \nabla\varphi_{\text{gauss}}(\mathbf{p}) = -2\varepsilon^2\,\varphi_{\text{gauss}}(\mathbf{p})\,\mathbf{p}. \tag{17}$$

Code correspondence.

- `rbf.anisotropic_diffusion_phi_cubic(p, A, tol)` $\leftrightarrow$ (15); `rbf.anisotropic_diffusion_phi_gauss(p, A, eps)` $\leftrightarrow$ (16).
- `rbf.anisotropic_diffusion_poly(p, A, tol)` returns `0.0`, matching the polynomial-basis observation above.
- `rbf.grad_phi_cubic(p)`, `rbf.grad_phi_gauss(p, eps)` $\leftrightarrow$ (17).
- `assembly.set_rbf_func` wires the chosen kernel, gradient, and anisotropic-diffusion closures (with A fixed) onto `context.phi`, `context.grad_phi`, and `context.laplacian_phi` respectively, dispatching on the string `basis` (`'cubic'` or `'gaussian'`).

5 Global assembly

5.1 Interior rows

For every node $\mathbf{x}_i$ in the domain, the local weight vector $\mathbf{w}^{(i)} \in \mathbb{R}^{|S_i|}$ computed by solving (9) (or (13)) is scattered into row i of a dense global matrix $W \in \mathbb{R}^{N\times N}$:

$$W_{ij} = \begin{cases} w_j^{(i)} & j \in S_i, \\ 0 & j \notin S_i. \end{cases} \tag{18}$$

At this stage, W approximates $\nabla\cdot(A\nabla\,\cdot)$ at every node, ignoring boundary conditions entirely.

Code correspondence. `assembly.global_weights(context, in_boundary, normal_vec)` loops over all stencils, dispatches to `local_weights_solve` or `local_weights_ls` depending on `context.center_rings`, and assembles (18).

5.2 Dirichlet rows

For a node $\mathbf{x}_i \in \Gamma_D$, row i of W is overwritten with the identity row,

$$W_{ij} = \delta_{ij}, \qquad i \in \Gamma_D, \tag{19}$$

so that the linear system enforces $u(\mathbf{x}_i) = g_D(\mathbf{x}_i)$ exactly (via the matching right-hand-side entry, Section 5.4).

5.3 Neumann rows

For a node $\mathbf{x}_i \in \Gamma_N$ with outward unit normal n_i, row i is overwritten with directional-derivative weights, built from the *local gradient* weight matrix $W^{(i)}_{\text{grad}} \in \mathbb{R}^{|S_i|\times 2}$ (the analog of (9) for $\mathcal{L} = \nabla$):

$$W_{ij} = \left(W^{(i)}_{\text{grad}}\, n_i\right)_j, \qquad j \in S_i, \quad i \in \Gamma_N, \tag{20}$$

so that row i of the assembled system enforces $n_i \cdot \nabla u(\mathbf{x}_i) \approx g_N(\mathbf{x}_i)$, matching (3).

Code correspondence. `assembly.boundary_to_weights` implements (19) and (20) in place, dispatching per node on the label returned by `in_boundary(p)` (itself built in `assembly.set_boundary_func` from `boundary.in_square_boundary` / `boundary.in_circular_boundary`, with normals from `boundary.square_normal` / `boundary.disk_normal`).

5.4 Right-hand side

The right-hand-side vector $\mathbf{F} \in \mathbb{R}^N$ is assembled nodewise,

$$F_i = \begin{cases} f(\mathbf{x}_i) & \mathbf{x}_i \text{ interior}, \\ g_D(\mathbf{x}_i) & \mathbf{x}_i \in \Gamma_D, \\ g_N(\mathbf{x}_i) & \mathbf{x}_i \in \Gamma_N, \end{cases} \tag{21}$$

so that the assembled linear system $W\mathbf{u} = \mathbf{F}$ is exactly the discrete analog of (1)–(3).

Code correspondence. `assembly.right_hand_side(context,f,g,in_boundary)` implements (21), where the single callable `g` passed in already dispatches internally to the correct side/arc function (see `boundary.square_boundary`, `boundary.circular_boundary`).

6 The pure-Neumann null space and anchoring

If $\Gamma_D = \emptyset$ (every boundary node is Neumann), the continuous problem (1)–(3) determines u only up to an additive constant, since $\nabla\cdot(A\nabla c) = 0$ and $n \cdot \nabla c = 0$ for any constant c. The discrete matrix W inherits this null space and is singular. Three regularization strategies are implemented:

$$\textbf{mean:}\ \ \textstyle\sum_i u_i = 0, \qquad \text{(replace the last row of } W \text{ by } \mathbf{1}^\top, \text{ set } F_{\text{last}} = 0), \tag{22}$$

$$\textbf{pin:}\ \ u_0 = 0, \qquad \text{(replace the first row of } W \text{ by } \mathbf{e}_0^\top, \text{ set } F_0 = 0), \tag{23}$$

$$\textbf{project:}\ \ W \leftarrow \Pi W \Pi, \quad \mathbf{F} \leftarrow \Pi\mathbf{F}, \qquad \Pi = I - \tfrac{1}{N}\mathbf{1}\mathbf{1}^\top. \tag{24}$$

Code correspondence. `assembly.is_pure_neumann(context,in_boundary)` detects this case; `assembly.anchor_system(W, f, method)` implements the three strategies in (24), selected by the string `method` (`'mean'`, `'pin'`, or `'project'`).

7 Solving and evaluating the discrete solution

The fully assembled (and, if necessary, anchored) linear system

$$W\mathbf{u} = \mathbf{F} \tag{25}$$

is solved directly for the nodal solution vector $\mathbf{u} \in \mathbb{R}^N$, approximating $u(\mathbf{x}_i)$ at every node.

Code correspondence. `assembly.rbf_fd_solve(W, F)` calls `np.linalg.solve(W, F)`, implementing (25). The full pipeline — nodes and stencils, kernel and boundary configuration, weight assembly, boundary imposition, right-hand-side assembly, and anchoring — is orchestrated end-to-end by `assembly.rbf_fd_system(...)`, which returns a populated `domain.PDEDomainContext` carrying A, W, and $\mathbf{F}$ for later use by `rbf_fd_solve`.

8 Convergence diagnostics

For manufactured-solution problems (Section 9), the discrete error is reported in both the maximum norm and a discrete L^2 norm:

$$\|e\|_\infty = \max_i |u_i - u_{\text{exact}}(\mathbf{x}_i)|, \qquad \|e\|_2 = \frac{1}{\sqrt{N}} \left(\sum_{i=1}^{N} \big(u_i - u_{\text{exact}}(\mathbf{x}_i)\big)^2 \right)^{1/2}, \tag{26}$$

together with the condition number $\kappa(W) = \|W\| \, \|W^{-1}\|$ as a diagnostic for ill-conditioning of the assembled system (which tends to grow as stencils shrink or as num_rings, ε, or node spacing are pushed to extremes).

Code correspondence. These are exactly the quantities computed and printed in `report_and_graph(context, u_exact)`: `np.linalg.cond(W)` for $\kappa(W)$, `np.max(error)` for $\|e\|_\infty$, and `np.linalg.norm(error)/np.sqrt(len(P))` for $\|e\|_2$, alongside contour and surface plots of the approximate solution, exact solution, and pointwise error.

9 Manufactured solutions

To validate the method, exact solutions u_{exact} are chosen in closed form, and the corresponding forcing term f and boundary data g_D, g_N are derived analytically so that u_{exact} satisfies (1)–(3) exactly (the *method of manufactured solutions*). For example, the mixed polynomial–trigonometric solution

$$u_{\text{exact}}(x,y) = (L-x)\Big(y - \tfrac{y^2}{L}\Big) + c\,(L-y)\,\frac{x(L-x)}{L} + \text{Amp}\,\sin\Big(\frac{\alpha\pi x}{L}\Big)\sin\Big(\frac{\beta\pi y}{L}\Big), \quad c = 2, \tag{27}$$

yields, after applying $-\nabla\cdot(A\nabla\,\cdot)$ to (27) and collecting polynomial, trigonometric, and mixed-derivative contributions, a forcing term f with three additive pieces:

$$f_{\text{poly}} = -2cA_{11}\frac{L-y}{L} + 2A_{12}\Big(\frac{2cx}{L} + \frac{2y}{L} - \frac{c+1}{L}\Big) - 2A_{22}\frac{L-x}{L}, \tag{28}$$

$$f_{\text{trig}} = -\text{Amp}\Big(A_{11}k_x^2 + A_{22}k_y^2\Big)\sin(k_x x)\sin(k_y y), \tag{29}$$

$$f_{\text{mixed}} = 2\,\text{Amp}\,A_{12}\,k_x k_y\,\cos(k_x x)\cos(k_y y), \tag{30}$$

with $k_x = \alpha\pi/L$, $k_y = \beta\pi/L$, and $f = f_{\text{poly}} + f_{\text{trig}} + f_{\text{mixed}}$, together with Dirichlet data read off directly from (27) on each edge.

Code correspondence. `example_3(Amp,modes,A,L)` returns the tuple `(f,g,btype,u_exact)` implementing exactly (27)–(30): `u_exact` evaluates (27), `f` evaluates the sum of (30), and `g` is the list of four one-variable boundary functions obtained by restricting (27) to each edge of the square.

10 Node generation strategies

The implementation supports several node distributions (`geometry.py`):

- **Uniform tensor-product grid** (`uniform_square`): a regular $N_x \times N_y$ Cartesian grid over $[0, L]^2$, boundary and interior nodes undifferentiated.
- **Uniform interior + explicit boundary** (`uniform_int_square`): interior nodes from a regular grid strictly inside $[0, L]^2$, concatenated with N_b equally spaced nodes per edge (corners de-duplicated); returns the interior node count alongside the array so that boundary/interior can be distinguished downstream if needed.
- **Chebyshev–Gauss–Lobatto grid** (`cheby_square`): tensor-product nodes $x_j = \frac{L}{2}\big(1 + \cos(j\pi/N_x)\big)$, clustering near the edges, as is standard for spectral collocation methods.
- **Circular domain** (`circular_geometry`): interior nodes from a Cartesian grid filtered to the open disk $x^2 + y^2 < R^2$, plus boundary nodes placed exactly on the circle at N_{bound} equally spaced angles.

In every case, the resulting node array $P \in \mathbb{R}^{N\times 2}$ is the sole input to `domain.PDEDomainContext`, after which stencils (`stencils.knn_list`) and RBF-FD weights are computed purely from the node positions, independent of how they were generated.